

Personal Task Manager

Web Application

Full-Stack SDE Project | Domain: SDE in IT

Team Members

Anantharaman • Bhuvana • Mohammed Ghouse •
Dhanushiya

Demo Date: 25 April 2026 • 3:00 PM



Problem Statement



Hard to Stay Organized

People struggle to track tasks, remember deadlines, and plan daily activities effectively.



Skill Gap for SDEs

Students practice coding problems but rarely build complete apps with frontend, backend & database.



No Real-World Experience

Working with APIs, databases, authentication & UI design is rarely taught end-to-end.

Goal: Build a professional Personal Task Manager where students learn full-stack development end-to-end.

Tech Stack & Architecture



Frontend

React.js

- ▶ React Components
- ▶ useState for state mgmt
- ▶ Axios for API calls
- ▶ Clean responsive UI
- ▶ Mark tasks done
- ▶ Filter & search tasks



Backend

Node.js + Express

- ▶ RESTful API endpoints
- ▶ CRUD operations
- ▶ Route handlers
- ▶ Error handling middleware
- ▶ CORS enabled
- ▶ dotenv config



Database

MongoDB Atlas

- ▶ Cloud-hosted free tier
- ▶ Mongoose ODM
- ▶ Task schema model
- ▶ title, description
- ▶ deadline, priority
- ▶ completed flag

API Design & CRUD Operations

Operation	Method	Endpoint	Description
CREATE	POST	<code>/api/tasks/add</code>	Add a new task to the database
READ	GET	<code>/api/tasks</code>	Retrieve all tasks, sorted by date
UPDATE	PUT	<code>/api/tasks/update/:id</code>	Modify task fields by ID
DELETE	DELETE	<code>/api/tasks/delete/:id</code>	Remove a task permanently

Task Model: { title, description, deadline, priority, completed, timestamps }

priority: enum['Low', 'Medium', 'High'] • completed: Boolean (default: false) • Auto createdAt / updatedAt

Week-Wise Development Plan

Week 1	Week 2	Week 3	Week 4
Backend API	Frontend UI	Extra Features	Demo & Present
Install Node.js & Express	React setup & components	Search bar feature	Create PPT presentation
Create folder structure	Navbar + Task List + Form	Filter by priority/deadline	Architecture diagram
Build all 4 CRUD routes	Axios API integration	Error handling messages	Live demo preparation
Connect MongoDB Atlas	Display tasks in cards	Light/Dark mode toggle	Q&A prep (API, CRUD...)
✓ Backend fully working	✓ Working UI with live data	✓ Polished production app	✓ Demo ready April 25

Start: 25 Mar 2026

End: 23 Apr 2026

Demo: 25 Apr 2026 • 3:00 PM

Key Concepts — Q&A Preparation

What is an API?

An Application Programming Interface lets two systems communicate. Like a waiter between you and the kitchen — you request, it fetches, it returns.

What is CRUD?

Create · Read · Update · Delete — the 4 core database operations. Every feature maps to one: Add Task = Create, View Tasks = Read, Edit = Update, Remove = Delete.

What is React State?

State is a component's memory. When state changes, the UI re-renders automatically. `useState([])` holds the task list; `setTasks()` triggers live updates.

Why Express.js?

Express wraps raw Node.js with clean routing, middleware support, and JSON handling. One line vs dozens of lines — faster, cleaner, industry-standard.

Live Demo Walkthrough

1 Add a Task

Fill in title, description, priority (High/Medium/Low) and deadline, then click Add.

2 View All Tasks

All tasks load from MongoDB Atlas via GET `/api/tasks` and display as cards.

3 Mark as Done

Click 'Mark Complete' — UI updates instantly; backend saves the completed flag.

4 Edit a Task

Click Edit → update any field → PUT `/api/tasks/update/:id` called automatically.

5 Delete a Task

Click Delete → confirmation → DELETE `/api/tasks/delete/:id` removes from DB.

6 Filter & Search

Search by keyword or filter by priority/deadline to find tasks instantly.

What We Learned

- ✓ Full-stack application architecture
- ✓ RESTful API design with Express
- ✓ MongoDB Atlas & Mongoose models
- ✓ React components & state management
- ✓ Axios for API integration
- ✓ Error handling & validation
- ✓ CORS, dotenv & environment config
- ✓ Real SDE workflow & code structure

Project Complete!



Backend API

All 4 CRUD endpoints working with MongoDB

React Frontend

Clean UI with live data from the API

Extra Features

Search, filter, dark mode, error handling

Demo Ready

Full presentation prepared for April 25

Thank You • Questions Welcome!